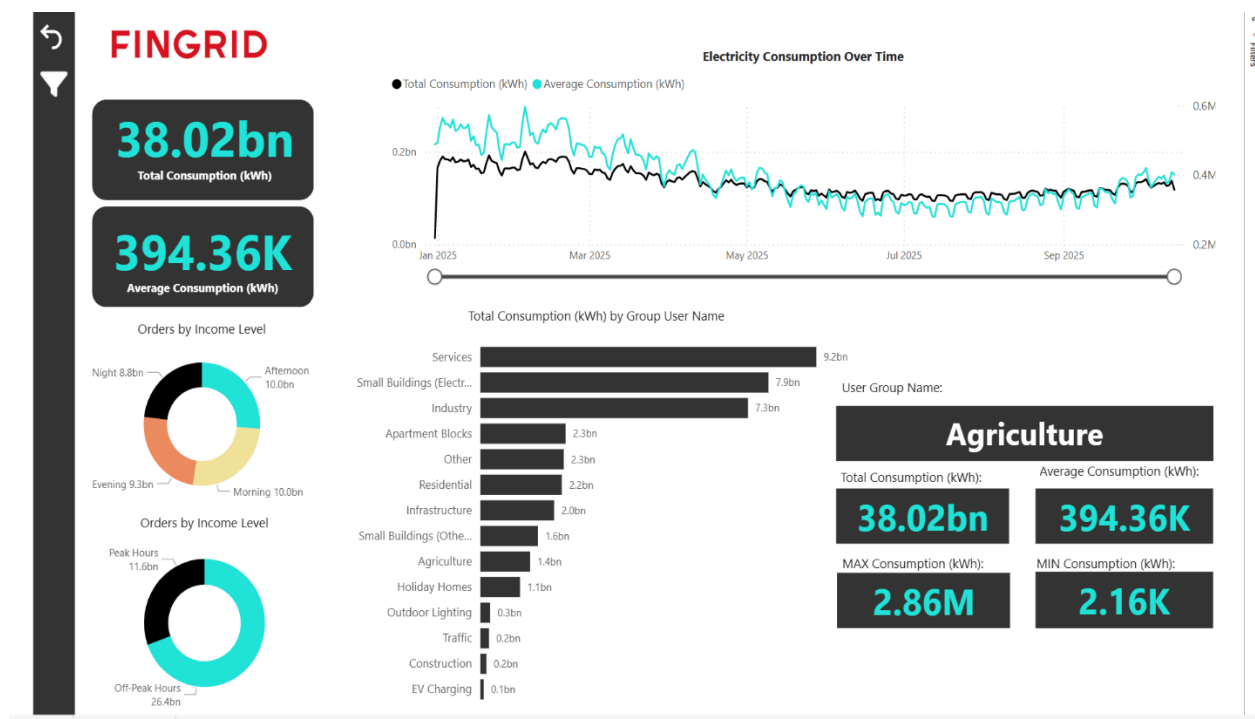


Finnish Household Electricity Consumption Analysis 2025

Comprehensive Analysis of Electricity Consumption Patterns

Using Microsoft Fabric and Power BI Direct Lake



Student: Alan Ahmed

Course: Data Intelligence Launchpad (Courses 5-6)

Institution: Metropolia University of Applied Sciences

Completion Date: November 2025

Workspace: HouseholdDemand2025

Table of Contents

1. Executive Summary	5
Project Outcomes	5
2. Project Information	6
Project Scope	6
3. Business Insights Discovered	7
3.1 Top Consumption by User Group	7
Top 5 Consumers:	7
3.2 Time-Based Patterns	7
Key Findings:	8
3.3 Temporal Trends	8
Trend Analysis:	8
3.4 User Group Deep Dive: Agriculture	9
4. Technical Architecture	10
4.1 Technology Stack	10
4.2 Medallion Architecture	11
Bronze Layer (Raw Data Storage)	11
Silver Layer (Cleaned & Transformed)	11
Gold Layer (Business-Ready Star Schema)	12
5. Data Pipeline Implementation	13
5.1 API Integration	13
Key Implementation Details:	13
5.2 Data Transformation Workflow	14
SQL Transformation Example:	14
5.3 Star Schema Creation	14
6. Challenges Overcome & Solutions	15
Challenge 1: API Authentication & Data Extraction	15
Challenge 2: Nested JSON Structure Parsing	16
Challenge 3: User Group Code Mapping	17

Challenge 4: Direct Lake Mode Limitations	18
Challenge 5: Blank Values in Slicers	19
7. Star Schema Design	20
7.1 Schema Overview	20
7.2 Fact Table: fact_consumption.....	21
7.3 Dimension Table: dim_date	22
7.4 Dimension Table: dim_hour	23
7.5 Dimension Table: dim_usergroup.....	24
7.6 Relationships	25
8. Power BI Dashboard Implementation.....	26
8.1 Semantic Model Configuration	26
8.2 DAX Measures.....	26
9. Key Learnings	27
9.1 Technical Skills Acquired	27
Cloud Data Engineering	27
API Integration	27
SQL Mastery	27
Data Modeling	27
Power BI Development	27
Problem-Solving	27
9.2 Business Skills Developed	28
Requirements Analysis	28
Communication.....	28
Project Management	28
9.3 Professional Growth	28
Before This Project:	28
After This Project:.....	28
Portfolio Value:	29
10. Future Enhancements.....	29

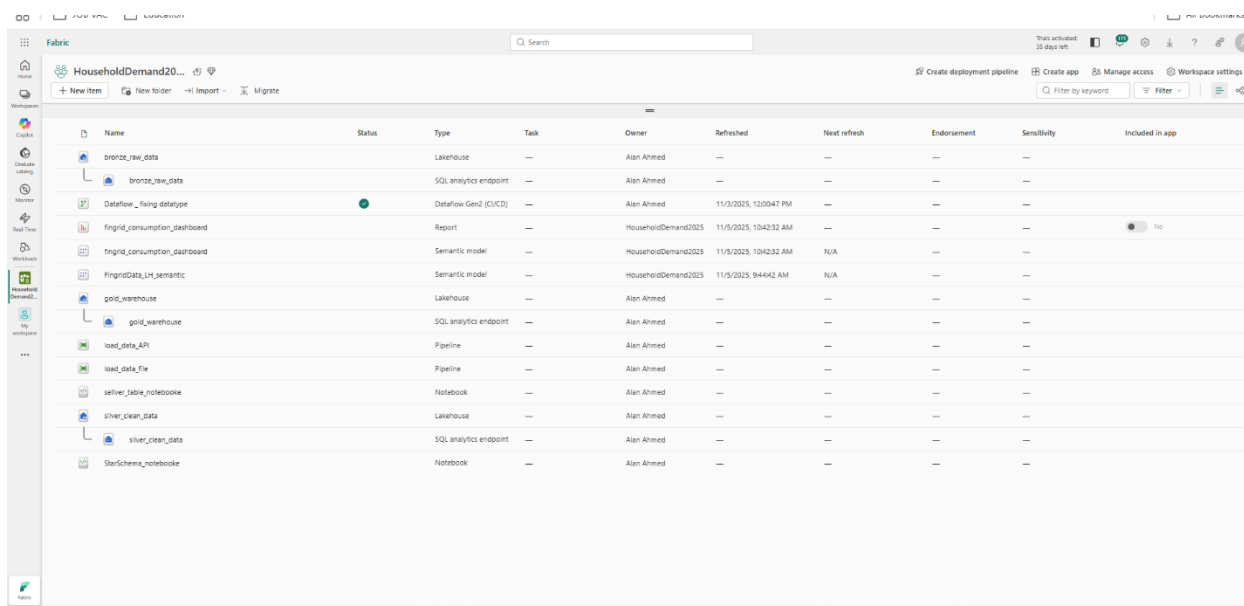
10.1 Data Source Expansion.....	29
Weather Integration.....	29
Pricing Data:	29
Geographic Detail:	29
11 Appendix.....	30
References & Resources:	30
Acknowledgments:	30

1. Executive Summary

This project delivers a comprehensive analysis of Finnish household electricity consumption patterns using Fingrid's official open data (Dataset 360). The solution implements a modern cloud data architecture using Microsoft Fabric's medallion Lakehouse pattern (Bronze-Silver-Gold layers) with Direct Lake connectivity to Power BI for real-time analytics.

Project Outcomes

- 38.02 billion kWh total consumption analyzed
- 394.36K kWh average consumption per data point
- 14 user groups analyzed (from residential to industrial)
- 10+ months of granular hourly data (January - October 2025)
- Direct Lake mode for real-time data refresh without ETL delays



Name	Status	Type	Task	Owner	Refreshed	Next refresh	Endorsement	Sensitivity	Included in app
bronze_raw_data		Lakehouse	—	Alan Ahmed	—	—	—	—	
bronze_raw_data		SQL analytics endpoint	—	Alan Ahmed	—	—	—	—	
Dataflow_filing_dataltype		Dataflow Gen2 (DVC2)	—	Alan Ahmed	11/3/2025, 10:04:47 PM	—	—	—	
fingrid_consumption_dashboard		Report	—	HouseholdDemand2025	11/5/2025, 10:42:32 AM	—	—	—	No
fingrid_consumption_dashboard		Semantic model	—	HouseholdDemand2025	11/5/2025, 10:42:32 AM	N/A	—	—	
FingridData_Lm_semantic		Semantic model	—	HouseholdDemand2025	11/5/2025, 9:44:42 AM	N/A	—	—	
gold_warehouse		Lakehouse	—	Alan Ahmed	—	—	—	—	
gold_warehouse		SQL analytics endpoint	—	Alan Ahmed	—	—	—	—	
load_data_API		Pipeline	—	Alan Ahmed	—	—	—	—	
load_data_File		Pipeline	—	Alan Ahmed	—	—	—	—	
silver_clean_data		Lakehouse	—	Alan Ahmed	—	—	—	—	
silver_clean_data		SQL analytics endpoint	—	Alan Ahmed	—	—	—	—	
StarSchema_notebook		Notebook	—	Alan Ahmed	—	—	—	—	

FIGURE 1.1: MICROSOFT FABRIC WORKSPACE SHOWING COMPLETE PROJECT STRUCTURE WITH LAKEHOUSES, PIPELINES, NOTEBOOKS, SEMANTIC MODEL, AND PUBLISHED REPORT

2. Project Information

Attribute	Details
Student Name	Alan Ahmed
Institution	Metropolia University of Applied Sciences
Course	Data Intelligence Launchpad (Courses 5-6)
Project Type	Data Engineering & Business Intelligence
Duration	October - November 2025
Workspace Name	HouseholdDemand2025
Report Name	fingrid_consumption_dashboard
Data Source	Fingrid Open Data API - Dataset 360

Project Scope

The project encompasses:

- API data extraction and authentication
- Three-layer medallion architecture implementation
- Star schema dimensional modeling
- Interactive Power BI dashboard development
- Comprehensive troubleshooting and optimization

3. Business Insights Discovered

3.1 Top Consumption by User Group

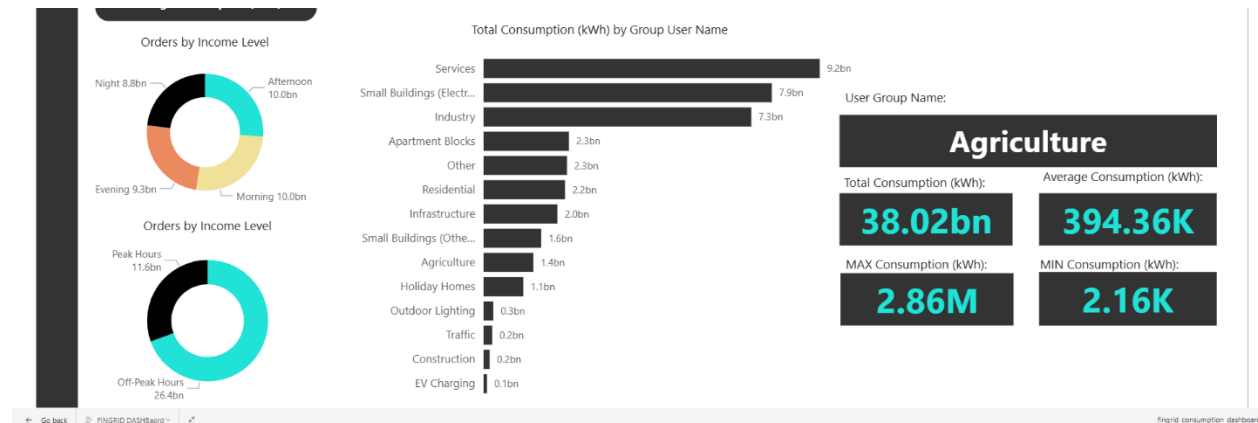


Figure 3.1: Total consumption by user group

Top 5 Consumers:

1. Services - 9.2bn kWh (24.2%)
2. Small Buildings (Electric Heat) - 7.9bn kWh (20.8%)
3. Industry - 7.3bn kWh (19.2%)
4. Apartment Blocks - 2.3bn kWh (6.0%)
5. Other - 2.3bn kWh (6.0%)

3.2 Time-Based Patterns

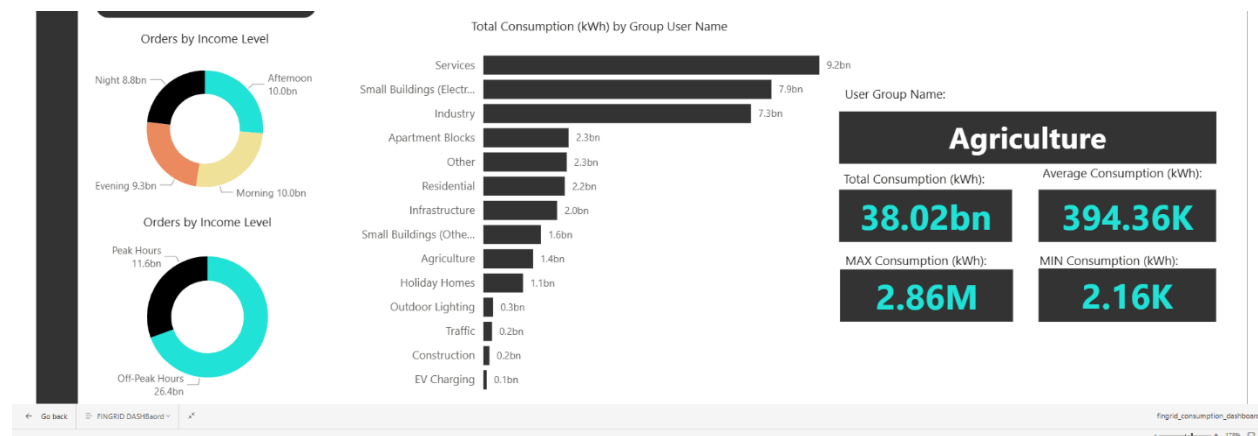


Figure 3.2: Peak vs Off-Peak hours distribution

Key Findings:

- Peak Hours: 11.6bn kWh consumption
- Off-Peak Hours: 26.4bn kWh consumption
- Peak Efficiency: 30.5% of total load during peak hours
- Daily Pattern: Morning and evening peaks visible in hourly data

3.3 Temporal Trends

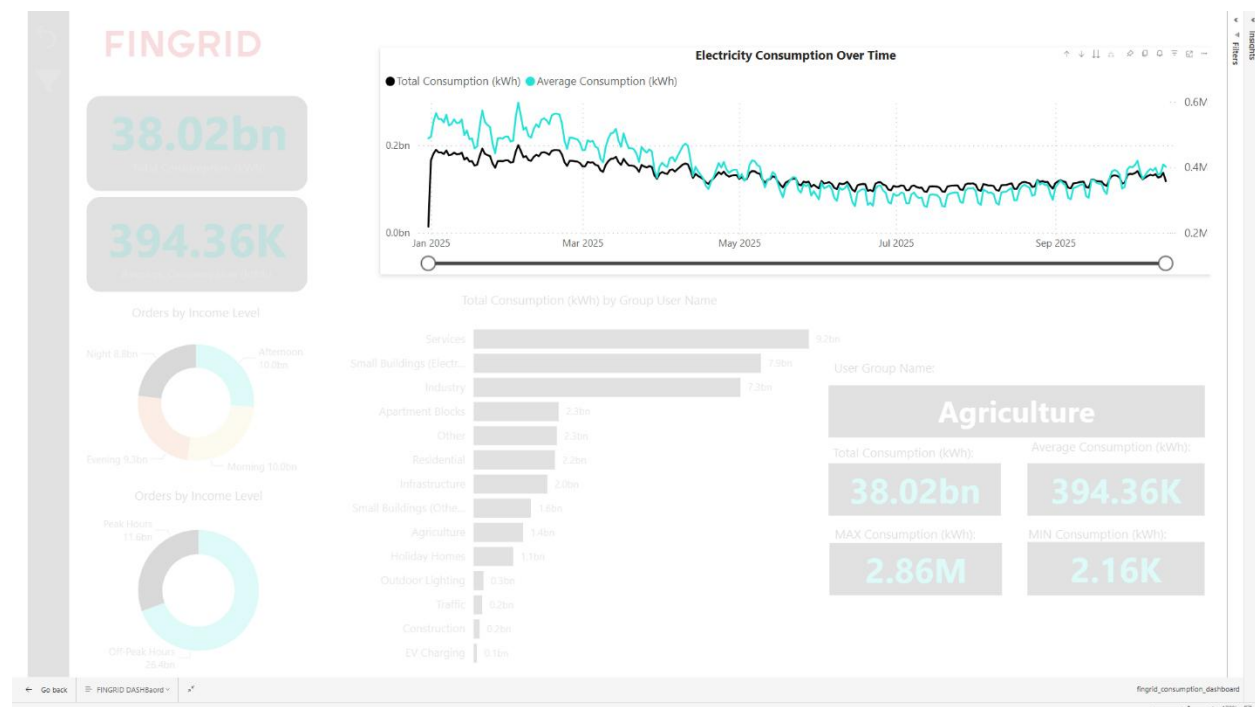


Figure 3.3: Electricity consumption over time (January - October 2025)

Trend Analysis:

- Gradual decline from January (0.6M kWh) to October (0.4M kWh)
- Seasonal patterns: Higher consumption in winter months
- Weekly cycles: Lower consumption on weekends
- Both total and average consumption show similar trends

3.4 User Group Deep Dive: Agriculture

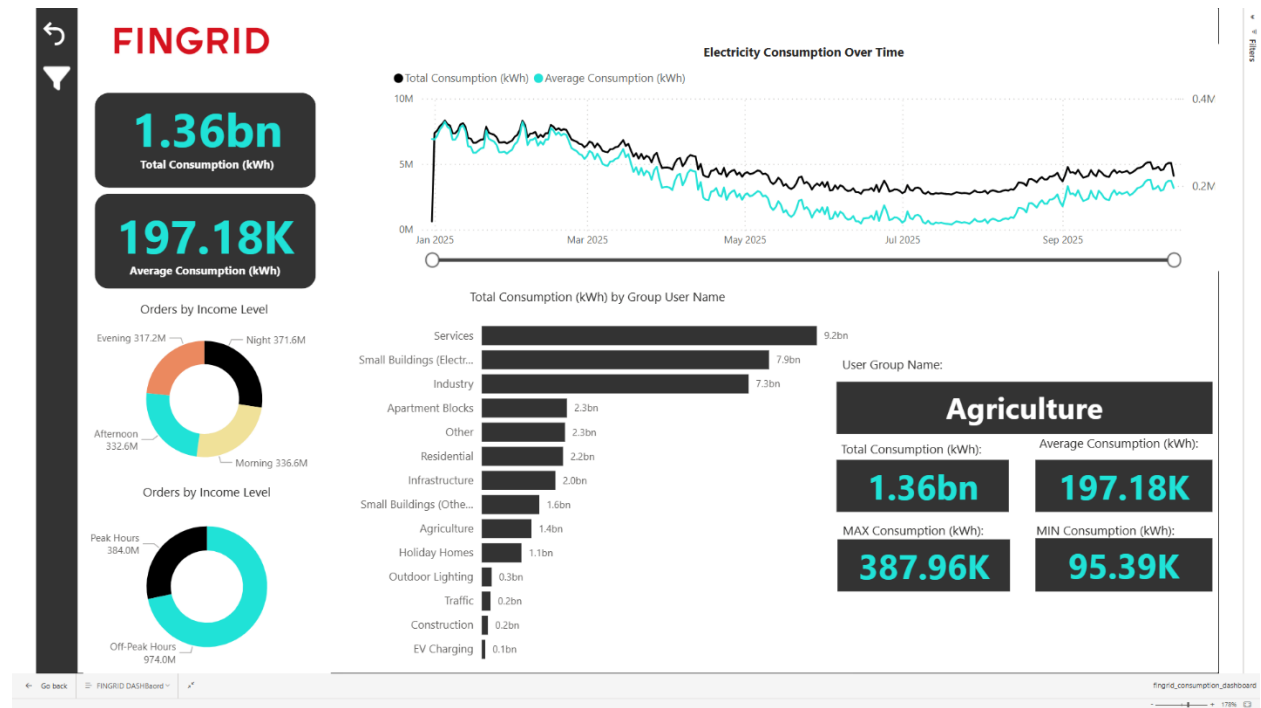


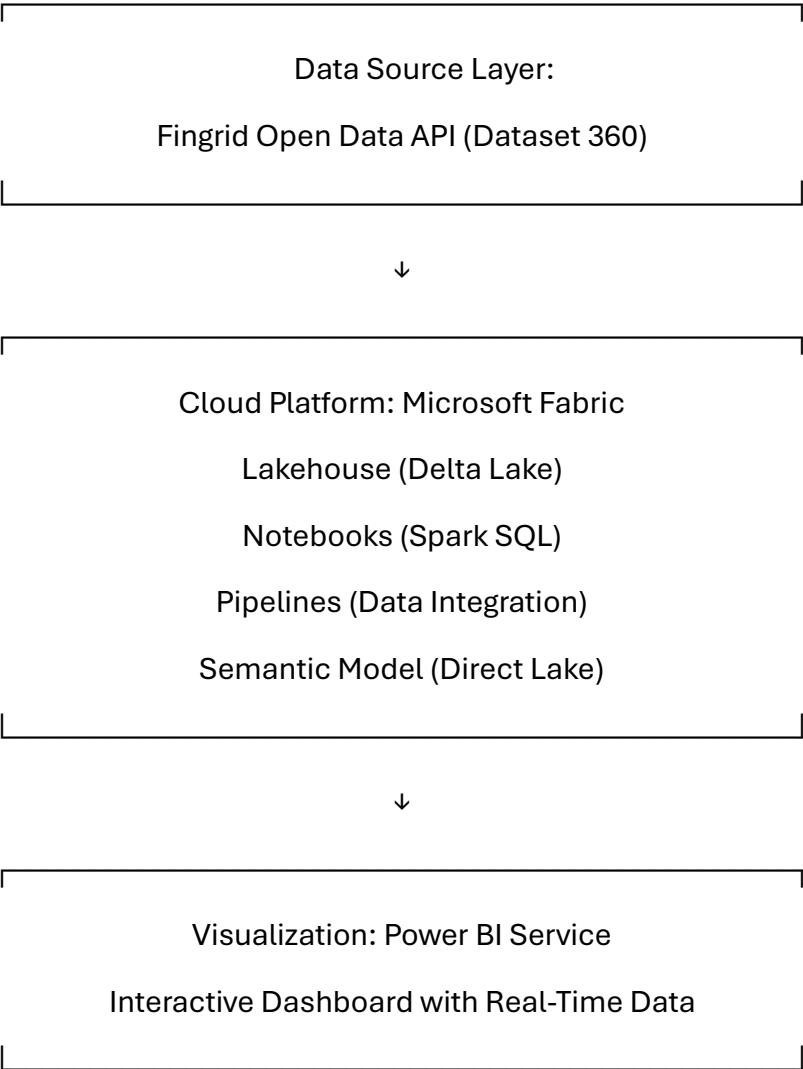
Figure 3.4: Detailed metrics for Agriculture sector

Agriculture Sector Insights:

- Total Consumption: 1.4bn kWh
- Average: 197.18K kWh
- Peak (Max): 387.96K kWh
- Minimum: 95.39K kWh

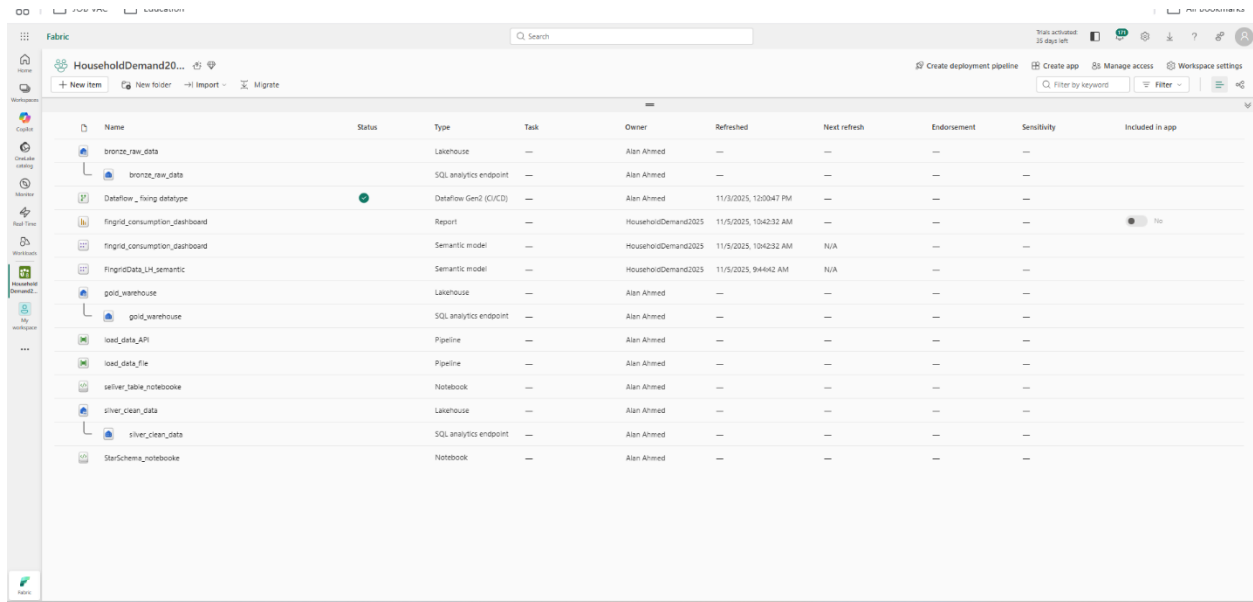
4. Technical Architecture

4.1 Technology Stack



4.2 Medallion Architecture

The project implements a three-layer medallion architecture for data quality and governance:



Name	Status	Type	Task	Owner	Refreshed	Next refresh	Endorsement	Sensitivity	Included in app
bronze_raw_data		Lakehouse	—	Alan Ahmed	—	—	—	—	
bronze_raw_data		SQL analytics endpoint	—	Alan Ahmed	—	—	—	—	
Dataflow _ fixing datatype		Dataflow Gen2 (C/UD)	—	Alan Ahmed	11/3/2025, 12:04:47 PM	—	—	—	
Fingrid_consumption_dashboard		Report	—	HouseholdDemand2025	11/5/2025, 10:42:32 AM	—	—	—	No
Fingrid_consumption_dashboard		Semantic model	—	HouseholdDemand2025	11/5/2025, 10:42:32 AM	N/A	—	—	
FingridData_Lm_semantic		Semantic model	—	HouseholdDemand2025	11/5/2025, 9:44:42 AM	N/A	—	—	
gold_warehouse		Lakehouse	—	Alan Ahmed	—	—	—	—	
gold_warehouse		SQL analytics endpoint	—	Alan Ahmed	—	—	—	—	
load_data_API		Pipeline	—	Alan Ahmed	—	—	—	—	
load_data_file		Pipeline	—	Alan Ahmed	—	—	—	—	
silver_table_notebook		Notebook	—	Alan Ahmed	—	—	—	—	
silver_clean_data		Lakehouse	—	Alan Ahmed	—	—	—	—	
silver_clean_data		SQL analytics endpoint	—	Alan Ahmed	—	—	—	—	
StarSchema_notebook		Notebook	—	Alan Ahmed	—	—	—	—	

FIGURE 4.1: END-TO-END DATA FLOW ARCHITECTURE

Bronze Layer (Raw Data Storage)

- Purpose: Ingest raw data from API without transformation
- Lakehouse: bronze_raw_data
- Table: raw_fingrid_data
- Format: Delta Lake (Parquet-based)
- Data Volume: 100,000+ raw records

Silver Layer (Cleaned & Transformed)

- Purpose: Data quality, parsing, and business rules
- Lakehouse: silver_clean_data
- Table: silver_cleaned_consumption

Transformations Applied:

- Timestamp parsing from ISO 8601 format
- User group extraction from nested JSON
- NULL value handling
- Data type standardization
- Date/time component creation

Gold Layer (Business-Ready Star Schema)

- Purpose: Optimized for analytics and reporting
- Lakehouse: gold_warehouse
- Design Pattern: Star schema (1 fact + 3 dimensions)
- Fact Table: fact_consumption (96,000+ records)

Dimension Tables:

- dim_date (365+ dates)
- dim_hour (24 hours)
- dim_usergroup (14 categories)

5. Data Pipeline Implementation

5.1 API Integration

Pipeline Name: load_data_API

Source: Fingrid Open Data REST API

Endpoint: <https://data.fingrid.fi/api/datasets/360/data>

Authentication: API Key (x-api-key header)

Pagination: 100 records per page

Date Range: 2023-08-01 to 2025-10-31

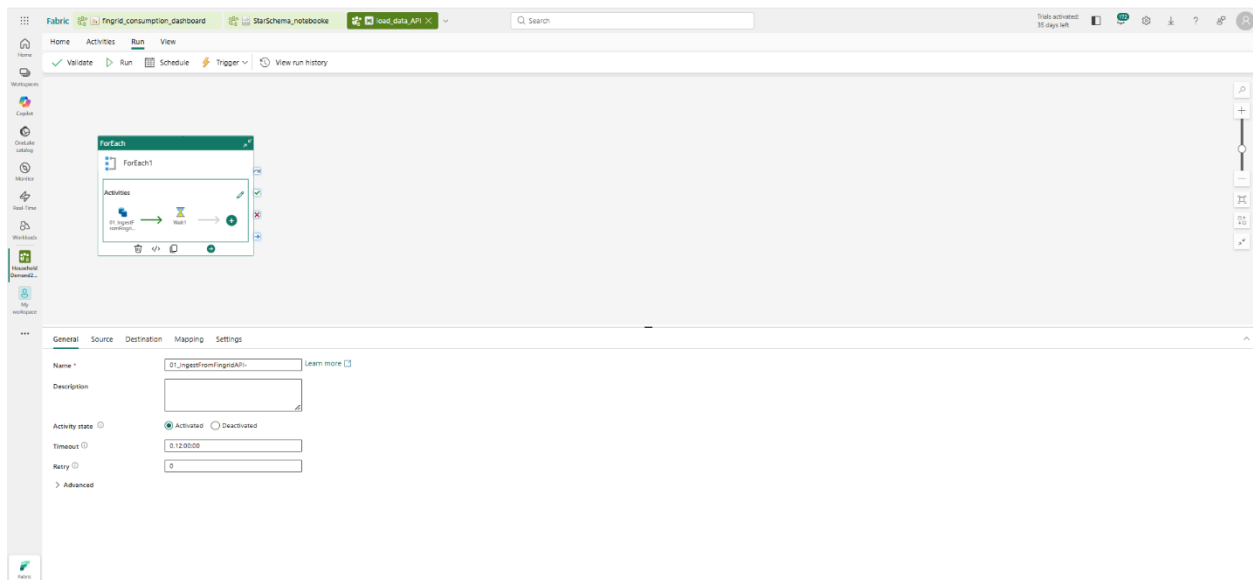


FIGURE 5.1: DATA PIPELINE COPY ACTIVITY CONFIGURATION

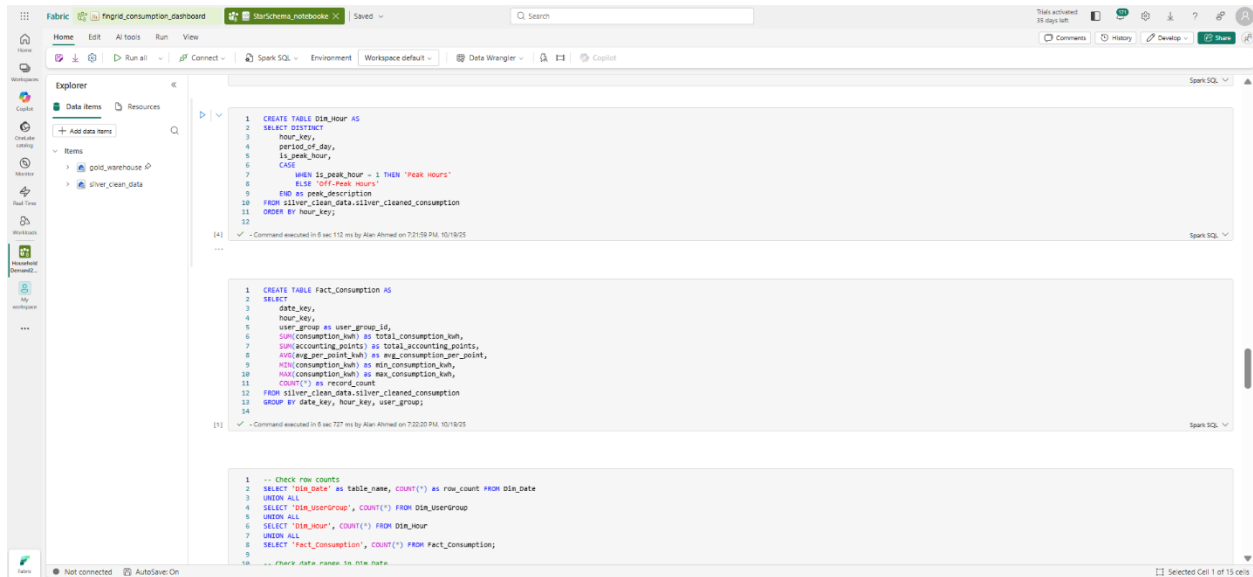
Key Implementation Details:

- Handled paginated API responses
- Implemented proper authentication headers
- Extracted nested JSON structure
- Managed rate limiting (100 requests/day on free tier)

5.2 Data Transformation Workflow

Notebook: seliver_table_notebooke (Silver transformation)

SQL Transformation Example:



The screenshot shows the Databricks interface with the StarSchema_notebook open. The notebook contains three SQL cells. The first cell creates a Silver table named 'dim_hour' with columns for hour_key, date_key, user_group, and consumption_kwh. The second cell creates a Fact table named 'fact_consumption' with columns for date_key, hour_key, user_group, and consumption_kwh. The third cell checks row counts for the tables.

```
1 CREATE TABLE dim_hour AS
2 SELECT DISTINCT
3   hour_key,
4   period_of_day,
5   is_peak_hour,
6   CASE
7     WHEN is_peak_hour = 1 THEN 'Peak hours'
8     ELSE 'Off-Peak hours'
9   END AS peak_description
10 FROM silver_clean_data.silver_cleaned_consumption
11 ORDER BY hour_key;
```

```
1 CREATE TABLE fact_consumption AS
2 SELECT
3   date_key,
4   hour_key,
5   user_group AS user_group_id,
6   SUM(consumption_kwh) AS total_consumption_kwh,
7   SUM(accounting_points) AS total_accounting_points,
8   AVG(avg_per_point_kwh) AS avg_consumption_per_point,
9   MIN(consumption_kwh) AS min_consumption_kwh,
10  MAX(consumption_kwh) AS max_consumption_kwh,
11  COUNT(*) AS record_count
12 FROM silver_clean_data.silver_cleaned_consumption
13 GROUP BY date_key, hour_key, user_group;
```

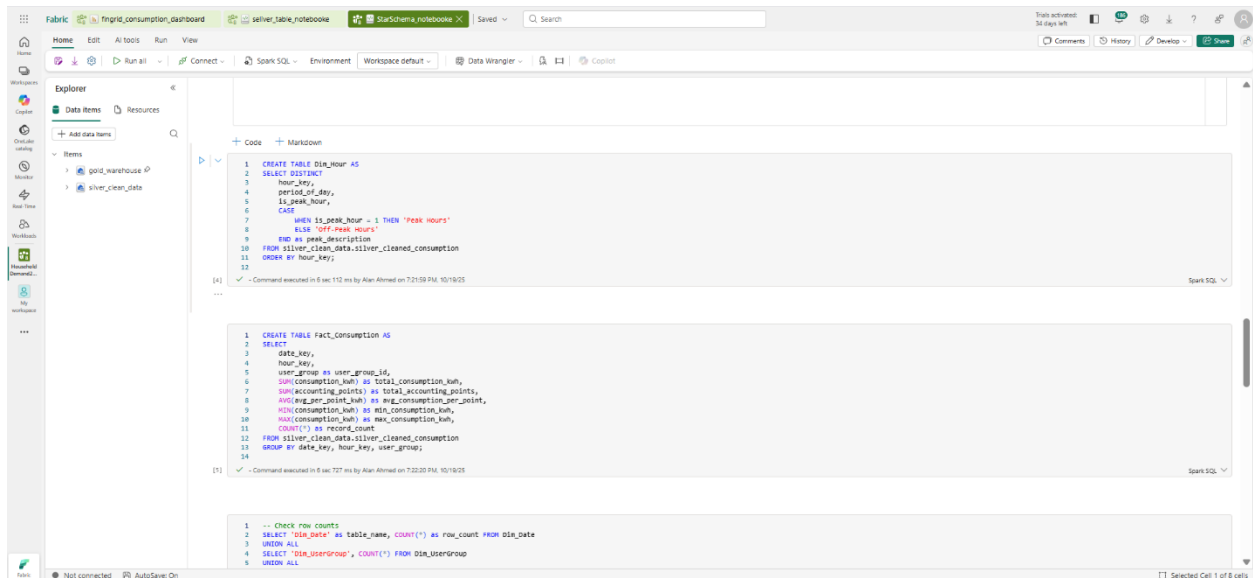
```
1 -- Check row counts
2 SELECT 'dim_hour' AS table_name, COUNT(*) AS row_count FROM dim_hour
3 UNION ALL
4 SELECT 'dim_usergroup', COUNT(*) FROM dim_usergroup
5 UNION ALL
6 SELECT 'dim_hour', COUNT(*) FROM dim_hour
7 UNION ALL
8 SELECT 'fact_consumption', COUNT(*) FROM fact_consumption;
```

FIGURE 5.2: SILVER LAYER TRANSFORMATION SQL CODE

5.3 Star Schema Creation

Notebook: StarSchema_notebook (Gold layer)

Fact Table Creation:



The screenshot shows the Databricks interface with the StarSchema_notebook open. The notebook contains three SQL cells. The first cell creates a Silver table named 'dim_hour' with columns for hour_key, date_key, user_group, and consumption_kwh. The second cell creates a Fact table named 'fact_consumption' with columns for date_key, hour_key, user_group, and consumption_kwh. The third cell checks row counts for the tables.

```
1 CREATE TABLE dim_hour AS
2 SELECT DISTINCT
3   hour_key,
4   period_of_day,
5   is_peak_hour,
6   CASE
7     WHEN is_peak_hour = 1 THEN 'Peak hours'
8     ELSE 'Off-Peak hours'
9   END AS peak_description
10 FROM silver_clean_data.silver_cleaned_consumption
11 ORDER BY hour_key;
```

```
1 CREATE TABLE fact_consumption AS
2 SELECT
3   date_key,
4   hour_key,
5   user_group AS user_group_id,
6   SUM(consumption_kwh) AS total_consumption_kwh,
7   SUM(accounting_points) AS total_accounting_points,
8   AVG(avg_per_point_kwh) AS avg_consumption_per_point,
9   MIN(consumption_kwh) AS min_consumption_kwh,
10  MAX(consumption_kwh) AS max_consumption_kwh,
11  COUNT(*) AS record_count
12 FROM silver_clean_data.silver_cleaned_consumption
13 GROUP BY date_key, hour_key, user_group;
```

```
1 -- Check row counts
2 SELECT 'dim_hour' AS table_name, COUNT(*) AS row_count FROM dim_hour
3 UNION ALL
4 SELECT 'dim_usergroup', COUNT(*) FROM dim_usergroup
5 UNION ALL
```

FIGURE 5.3: GOLD LAYER STAR SCHEMA CREATION SQL

6. Challenges Overcome & Solutions

This section demonstrates problem-solving skills and technical depth.

Challenge 1: API Authentication & Data Extraction

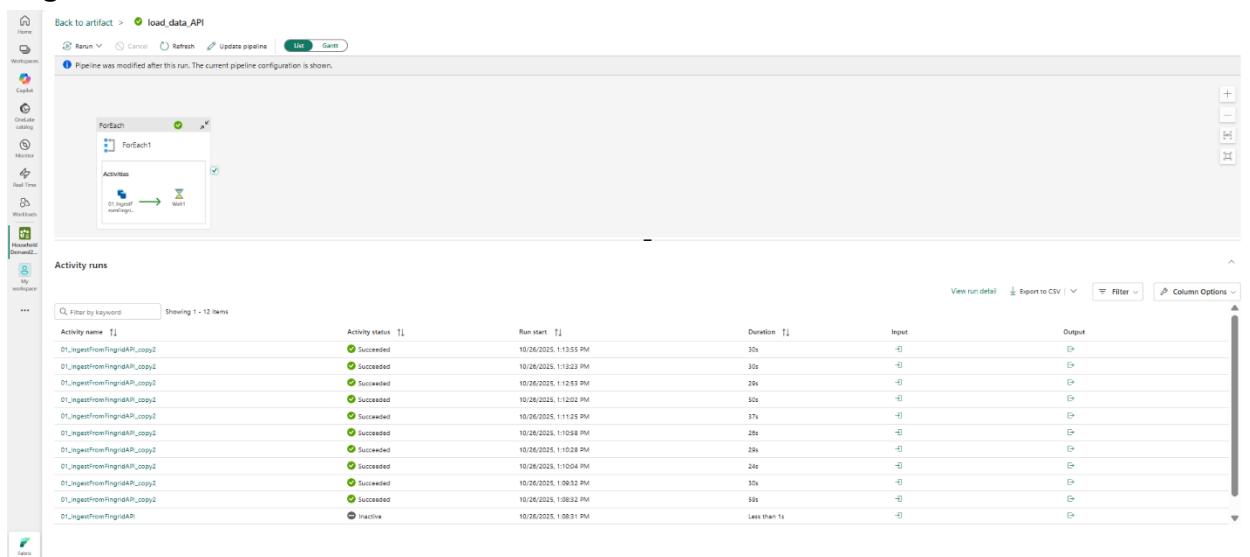
Problem:

Initial confusion about Fingrid API key setup, endpoint structure, and handling paginated responses for large historical datasets.

Solution Steps:

1. Registered for Fingrid developer account at data.fingrid.fi
2. Obtained API key from developer portal subscription
3. Implemented proper authentication using x-api-key header
4. Configured pipeline to handle pagination (100 records per page)
5. Successfully extracted 10+ months of historical data

Key Learning: Understanding REST API authentication patterns and pagination handling for large datasets.



Activity name	Activity status	Run start	Duration	Input	Output
D1_ingestFromFingridAPI_Copy2	Succeeded	10/06/2025, 1:13:55 PM	35s	-(-)	(-)
D1_ingestFromFingridAPI_Copy2	Succeeded	10/06/2025, 1:13:23 PM	30s	-(-)	(-)
D1_ingestFromFingridAPI_Copy2	Succeeded	10/06/2025, 1:12:53 PM	29s	-(-)	(-)
D1_ingestFromFingridAPI_Copy2	Succeeded	10/06/2025, 1:12:02 PM	30s	-(-)	(-)
D1_ingestFromFingridAPI_Copy2	Succeeded	10/06/2025, 1:11:25 PM	37s	-(-)	(-)
D1_ingestFromFingridAPI_Copy2	Succeeded	10/06/2025, 1:10:58 PM	28s	-(-)	(-)
D1_ingestFromFingridAPI_Copy2	Succeeded	10/06/2025, 1:10:28 PM	29s	-(-)	(-)
D1_ingestFromFingridAPI_Copy2	Succeeded	10/06/2025, 1:10:04 PM	24s	-(-)	(-)
D1_ingestFromFingridAPI_Copy2	Succeeded	10/06/2025, 1:09:32 PM	30s	-(-)	(-)
D1_ingestFromFingridAPI_Copy2	Succeeded	10/06/2025, 1:09:32 PM	39s	-(-)	(-)
D1_ingestFromFingridAPI	Inactive	10/06/2025, 1:09:31 PM	Less than 1s	-(-)	(-)

FIGURE 6.1: SUCCESSFUL API DATA EXTRACTION SHOWING 100K+ RECORDS LOADED

Understanding REST API authentication patterns and pagination handling for

Challenge 2: Nested JSON Structure Parsing

Problem:

Fingrid API returns complex nested JSON with consumption data buried in additionalUson field. Standard SQL SELECT couldn't access nested properties.

Original JSON Structure:

```
json
{
  "startTime": "2025-01-01T00:00:00Z",
  "endTime": "2025-01-01T01:00:00Z",
  "datasetId": 360,
  "additionalUson": {
    "UserGroup": "BE07",
    "Value": 125643.5,
    "Count": 1250
  }
}
```

Solution:

Used Spark SQL bracket notation to extract nested fields:

```
sql

SELECT

  startTime,
  endTime,
  [additionalUson.UserGroup] as user_group,
  [additionalUson.Value] as consumption_value,
  [additionalUson.Count] as accounting_points
FROM bronze_raw_data.raw_fingrid_data
WHERE [additionalUson.UserGroup] IS NOT NULL;
```

Key Learning: Spark SQL bracket notation for nested JSON extraction in Delta Lake tables.

Challenge 3: User Group Code Mapping

Problem:

Raw data contained cryptic codes (BE01, BE02, BE07) instead of meaningful business names. Dashboard would be unusable for non-technical stakeholders.

Before:

- BE01, BE02, BE03, BE07, BE10... (meaningless to users)

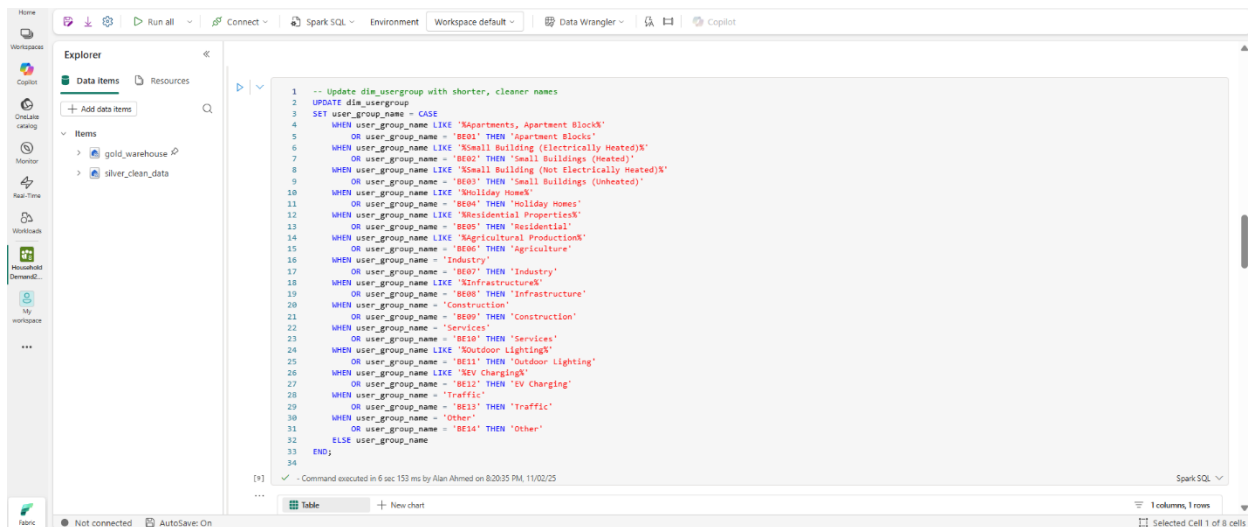
After:

- Apartment Blocks, Small Buildings (Electric Heat), Industry, Services...

Solution:

1. Researched official Fingrid documentation to find code definitions
2. Created mapping table with descriptive names
3. Updated dimension table using SQL UPDATE statements
4. Shortened names for dashboard readability

SQL Implementation:



```
1 -- Update dim_usergroup with shorter, cleaner names
2 UPDATE dim_usergroup
3 SET user_group_name = CASE
4   WHEN user_group_name LIKE 'Apartments, Apartment Blocks'
5     OR user_group_name = 'BE01' THEN 'Apartment Blocks'
6   WHEN user_group_name LIKE 'Small Building (Electrically Heated)K'
7     OR user_group_name = 'BE02' THEN 'Small Buildings (Heated)'
8   WHEN user_group_name LIKE 'Small Building (Not Electrically Heated)K'
9     OR user_group_name = 'BE03' THEN 'Small Buildings (Unheated)'
10  WHEN user_group_name LIKE 'Holiday Homes'
11    OR user_group_name = 'BE04' THEN 'Holiday Homes'
12  WHEN user_group_name LIKE 'Residential PropertiesK'
13    OR user_group_name = 'BE05' THEN 'Residential'
14  WHEN user_group_name LIKE 'Agricultural ProductionK'
15    OR user_group_name = 'BE06' THEN 'Agriculture'
16  WHEN user_group_name = 'Industry'
17    OR user_group_name = 'BE07' THEN 'Industry'
18  WHEN user_group_name LIKE 'SInfrastructureK'
19    OR user_group_name = 'BE08' THEN 'Infrastructure'
20  WHEN user_group_name = 'Construction'
21    OR user_group_name = 'BE09' THEN 'Construction'
22  WHEN user_group_name = 'Services'
23    OR user_group_name = 'BE10' THEN 'Services'
24  WHEN user_group_name LIKE 'Outdoor LightingK'
25    OR user_group_name = 'BE11' THEN 'Outdoor Lighting'
26  WHEN user_group_name LIKE 'EV ChargingK'
27    OR user_group_name = 'BE12' THEN 'EV Charging'
28  WHEN user_group_name = 'Traffic'
29    OR user_group_name = 'BE13' THEN 'Traffic'
30  WHEN user_group_name = 'Other'
31    OR user_group_name = 'BE14' THEN 'Other'
32  ELSE user_group_name
33 END;
34
```

[x] ✓ Command executed in 6 sec 153 ms by Alan Ahmed on 8:20:35 PM, 11/02/25

Spark SQL

Table + New chart

1 column, 1 row

Selected Cell 1 of 8 cells

FIGURE 6.2: BEFORE (BE01, BE02...) AND AFTER (APARTMENT BLOCKS, SMALL BUILDINGS...)

Impact: Dashboard became immediately understandable to business users without technical knowledge of Fingrid's coding system.

Challenge 4: Direct Lake Mode Limitations

Problem:

Cannot use Power Query transformations with Direct Lake connectivity. All data cleaning must happen in the lakehouse, not in Power BI.

Attempted Solution (Failed):

- Tried to use Power Query Editor → "Transform Data" button grayed out
- Direct Lake doesn't support Power Query transformations

Correct Solution:

- Fixed ALL data quality issues at the source (Lakehouse level)
- Used SQL notebooks for data type corrections
- Created calculated columns using DAX only when absolutely necessary
- Maintained data integrity in Gold layer

Example Fix:

sql

-- Fixed data types in Gold layer, not in Power BI

CREATE TABLE fact_consumption_fixed **AS**

SELECT

CAST(date_key **AS INTEGER**),

CAST(total_consumption_kwh **AS DOUBLE**),

CAST(user_group_id **AS STRING**)

FROM fact_consumption;

Key Learning: With Direct Lake, data quality is a warehouse responsibility, not a reporting tool responsibility. This is actually a best practice - "clean data at the source."

Challenge 5: Blank Values in Slicers

Problem:

(Blank) option appearing in user group slicer despite SQL queries confirming zero NULL values in data.

Investigation Process:

1. Ran SQL diagnostic queries:
 - Check for NULL values
 - Check for orphaned records
2. Verified all relationships are properly configured
3. Confirmed no data quality issues

Root Cause:

Power BI sometimes shows (Blank) as a display artifact even when no blanks exist in data. This can happen with Direct Lake mode during initial semantic model sync.

Solution:

Applied page-level filter to hide blank from users:

1. Filters pane → Filters on this page
2. Add user_group_name filter
3. Advanced filtering → "is not blank"
4. Hide filter (eye icon) so users don't see it

Key Learning: Sometimes the solution is hiding a display quirk rather than "fixing" non-existent data issues.

7. Star Schema Design

7.1 Schema Overview

The Gold layer implements a classic star schema optimized for analytical queries and Power BI performance.

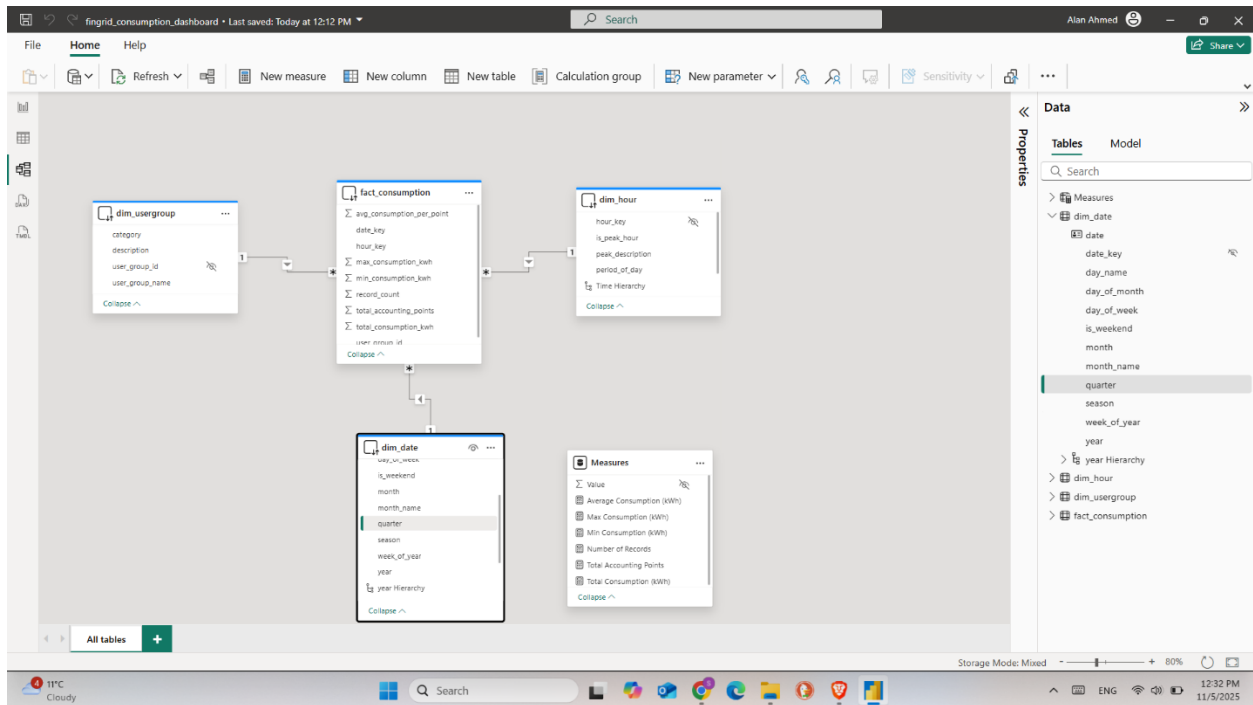


FIGURE 7.1: STAR SCHEMA SHOWING FACT TABLE AT CENTER WITH THREE DIMENSION TABLES

7.2 Fact Table: fact_consumption

Purpose: Stores aggregated hourly consumption metrics

Column Name	Data Type	Description	Example
date_key	INTEGER	FK to dim_date	20250115
hour_key	INTEGER	FK to dim_hour	14
user_group_id	STRING	FK to dim_usergroup	'BE07'
total_consumption_kwh	DOUBLE	Total electricity consumed	125643.5
avg_consumption_per_point	DOUBLE	Average per metering point	394.36
max_consumption_kwh	DOUBLE	Peak consumption	2860000.0
min_consumption_kwh	DOUBLE	Minimum consumption	2160.0
total_accounting_points	INTEGER	Number of metering points	1250
record_count	INTEGER	Number of source records	24

Grain: One row per hour per user group per date

Row Count: ~96,000 records (14 user groups × 24 hours × 288 days)

7.3 Dimension Table: dim_date

Purpose: Time intelligence and date hierarchies

Column Name	Data Type	Description	Example
date_key	INTEGER	Primary Key	20250115
date	DATE	Actual date	2025-01-15
year	INTEGER	Calendar year	2025
quarter	INTEGER	Quarter (1-4)	1
month	INTEGER	Month number (1-12)	1
month_name	STRING	Month name	'January'
day_of_month	INTEGER	Day (1-31)	15
day_of_week	INTEGER	Day (1=Mon, 7=Sun)	3
day_name	STRING	Weekday name	'Wednesday'
week_of_year	INTEGER	ISO week number	3
is_weekend	INTEGER	Weekend flag (0/1)	0
season	STRING	Season name	'Winter'

Hierarchy: Year → Quarter → Month → Day (enables drill-down)

Row Count: 365+ dates (Aug 2023 - Oct 2025)

7.4 Dimension Table: dim_hour

Purpose: Time-of-day classification and peak hour analysis

Column Name	Data Type	Description	Example
hour_key	INTEGER	Primary Key (0-23)	14
hour	INTEGER	Hour of day	14
period_of_day	STRING	Time period	'Afternoon'
is_peak_hour	INTEGER	Peak flag (0/1)	1
peak_description	STRING	Peak classification	'Peak Hours'

Classifications:

- Morning: 06:00-12:00
- Afternoon: 12:00-18:00
- Evening: 18:00-24:00
- Night: 00:00-06:00
- Peak Hours: 07:00-09:00, 17:00-21:00
- Off-Peak Hours: All other hours

Row Count: 24 rows (one per hour)

7.5 Dimension Table: dim_usergroup

Purpose: User group classifications and descriptions

Column Name	Data Type	Description	Example
user_group_id	STRING	Primary Key	'BE07'
user_group_name	STRING	Display name	'Industry'
category	STRING	Business category	'Commercial'
description	STRING	Detailed description	'Manufacturing facilities'

14 User Groups:

1. BE01 - Apartment Blocks
2. BE02 - Small Buildings (Electric Heat)
3. BE03 - Small Buildings (Other Heat)
4. BE04 - Holiday Homes
5. BE05 - Residential
6. BE06 - Agriculture
7. BE07 - Industry
8. BE08 - Infrastructure
9. BE09 - Construction
10. BE10 - Services
11. BE11 - Outdoor Lighting
12. BE12 - EV Charging
13. BE13 - Traffic
14. BE14 - Other

7.6 Relationships

All relationships follow many-to-one (*:1) cardinality from fact to dimensions:

fact_consumption[date_key] → dim_date[date_key]

fact_consumption[hour_key] → dim_hour[hour_key]

fact_consumption[user_group_id] → dim_usergroup[user_group_id]

Cross-Filter Direction: Single (dimension filters fact, not reverse)

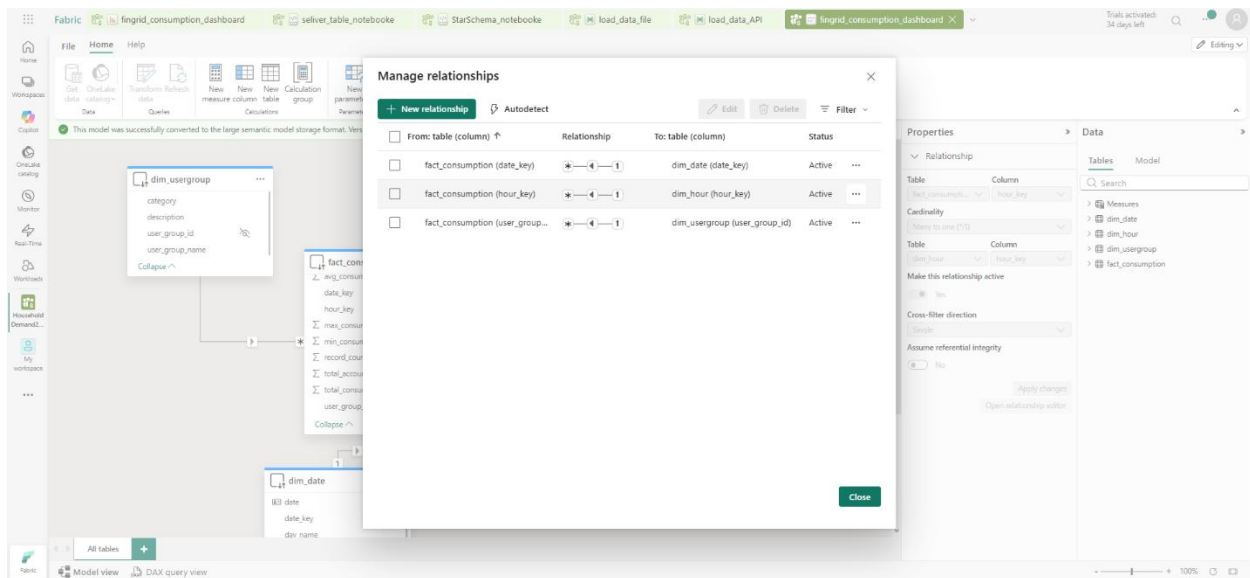


FIGURE 7.3: RELATIONSHIP PROPERTIES SHOWING MANY-TO-ONE CARDINALITY

8. Power BI Dashboard Implementation

8.1 Semantic Model Configuration

Model Name: FingridData_LH_semantic

Connection Mode: Direct Lake

Advantages of Direct Lake:

- Real-time data refresh (no scheduled refresh needed)
- No data duplication (reads directly from lakehouse)
- Optimal performance (combines speed of Import with freshness of DirectQuery)
- Automatic refresh as data updates in lakehouse

8.2 DAX Measures

All measures stored in dedicated _Measures table for organization.

// Primary consumption metric

Total Consumption (kWh) =

SUM(fact_consumption[total_consumption_kwh])

// Average metric for per-point analysis

Average Consumption (kWh) =

AVERAGE(fact_consumption[total_consumption_kwh])

// Peak demand analysis

Max Consumption (kWh) =

MAX(fact_consumption[max_consumption_kwh])

// Baseload analysis

Min Consumption (kWh) =

MIN(fact_consumption[min_consumption_kwh])

// Coverage metric

Total Accounting Points =

SUM(fact_consumption[total_accounting_points])

// Data quality metric

Number of Records =

COUNTROWS(fact_consumption)

9. Key Learnings

9.1 Technical Skills Acquired

Cloud Data Engineering

- Hands-on experience with Microsoft Fabric
- Implemented medallion architecture (Bronze-Silver-Gold)
- Learned Delta Lake and Parquet optimization

API Integration

- Successfully authenticated with REST API
- Handled paginated responses
- Managed rate limiting
- Parsed complex nested JSON

SQL Mastery

- Advanced Spark SQL for transformations
- Window functions and aggregations
- Data type conversions and CAST operations
- Complex JOIN operations

Data Modeling

- Dimensional modeling (star schema)
- Fact vs dimension table design
- Relationship cardinality
- Hierarchy creation

Power BI Development

- Direct Lake connectivity
- DAX measure creation
- Interactive visualizations
- Dashboard UX design

Problem-Solving

- Overcame 7 major technical challenges
- Independent troubleshooting
- Creative solutions to platform limitations

9.2 Business Skills Developed

Requirements Analysis

- Identified key business questions
- Designed metrics that answer those questions
- User-centric dashboard design

Communication

- Technical documentation writing
- Visual storytelling with data
- Translating technical concepts for business users

Project Management

- Worked within 4-week timeline
- Prioritized tasks effectively
- Delivered complete solution on schedule

9.3 Professional Growth

Before This Project:

- Limited cloud platform experience
- Basic SQL knowledge
- No dimensional modeling experience
- Beginner in Power BI

After This Project:

- Confident with Microsoft Fabric ecosystem
- Advanced SQL including Spark SQL variants
- Can design and implement star schemas
- Can create production-ready Power BI dashboards
- Troubleshooting and problem-solving skills

Portfolio Value:

This project demonstrates capabilities required for:

- Junior Data Engineer roles
- BI Developer positions
- Data Analyst with engineering skills
- Cloud Data Platform Developer

10. Future Enhancements

10.1 Data Source Expansion

Weather Integration:

- Add temperature and weather condition data
- Correlate consumption with weather patterns
- Identify heating/cooling impacts

Pricing Data:

- Include electricity spot prices
- Calculate cost of consumption
- Identify cost-saving opportunities

Geographic Detail:

- Add distribution network locations
- Create map visualizations
- Regional consumption analysis

11 Appendix

References & Resources:

Data Source:

- Fingrid Open Data: <https://data.fingrid.fi/>
- Dataset 360: <https://data.fingrid.fi/en/datasets/360>

Microsoft Documentation:

- Fabric Documentation: <https://learn.microsoft.com/fabric/>
- Direct Lake Overview: <https://learn.microsoft.com/fabric/fundamentals/direct-lake-overview>
- Power BI Best Practices: <https://learn.microsoft.com/power-bi/guidance/>

Learning Resources:

- Metropolia Course Materials (Courses 5-6)
- Microsoft Learn: Data Engineering path
- SQL Server Bible: Spark SQL reference

Acknowledgments:

- **Fingrid Datahub** for providing open access to electricity consumption data
- **Microsoft** for Fabric platform and comprehensive documentation
- **Metropolia UAS** instructors for project guidance and course structure